HOS08A — CI/CD

Deploying a Static Website to Azure with a GitHub Actions Pipeline

Mohammad Ra’uf Naser Al Batayneh
CS504 — Software Engineering
School of Technology & Computing, City University of Seattle
August 23, 2026

Item	Value
Live site	https://ashy-mushroom-075b8701e.7.azurestaticapps.net
Source repository	https://github.com/M7MDRAUF/HOS08
Azure resource	Static Web App "HOS08" in resource group "HOS08_group"
Hosting plan	Free
Region (Functions API / staging)	West US 2
Pipeline runs	4 of 4 succeeded

1. Introduction

This hands-on exercise builds a continuous integration and continuous deployment (CI/CD) pipeline for a static website hosted on Azure Static Web Apps. The objective is not simply to put a page on the internet — it is to demonstrate that a change committed to a feature branch reaches production automatically, with the pipeline as the only path to the live site.

The work is organised in three parts. First, a static website is created in a GitHub repository. Second, an Azure Static Web App is provisioned and connected to that repository, at which point Azure generates a GitHub Actions workflow and commits it to the default branch. Third, the pipeline is exercised end to end: a change is made on a "development" branch, submitted as a pull request, and merged into "main", and the deployed site is checked to confirm the change arrived without any manual deployment step.

1.1 Learning outcomes addressed

- Deploy a static website on the Azure platform.
- Build and operate a CI/CD pipeline using GitHub Actions.

1.2 Configuration used

Every value below was applied exactly as specified in the assignment handout.

Setting	Value
Subscription	Azure for Students Starter
Resource group	HOS08_group (new)
Static Web App name	HOS08
Plan type	Free — for hobby or personal projects
Region for Functions API / staging	West US 2
Deployment source	GitHub
Organization	M7MDRAUF
Repository	HOS08
Branch	main
Build preset	HTML
App location	/
Api location	(empty)
Output location	/

The Free plan was selected deliberately. The Azure portal defaults this control to "Standard", which is a billable tier; leaving the default in place would have incurred charges.

2. Part A — Creating the Static Website

A public GitHub repository named HOS08 serves as the single source of truth for the site. The repository holds one page, index.html, containing a single level-one heading.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>HOS08 — Azure Static Web Apps CI/CD</title>
</head>
<body>
    <h1>Hello</h1>
</body>
</html>
```

The page was committed and pushed to the main branch. Pushing before creating the Azure resource matters: Azure reads the repository during provisioning to detect the project structure and to offer the branch list, and an empty repository has no branches to offer.

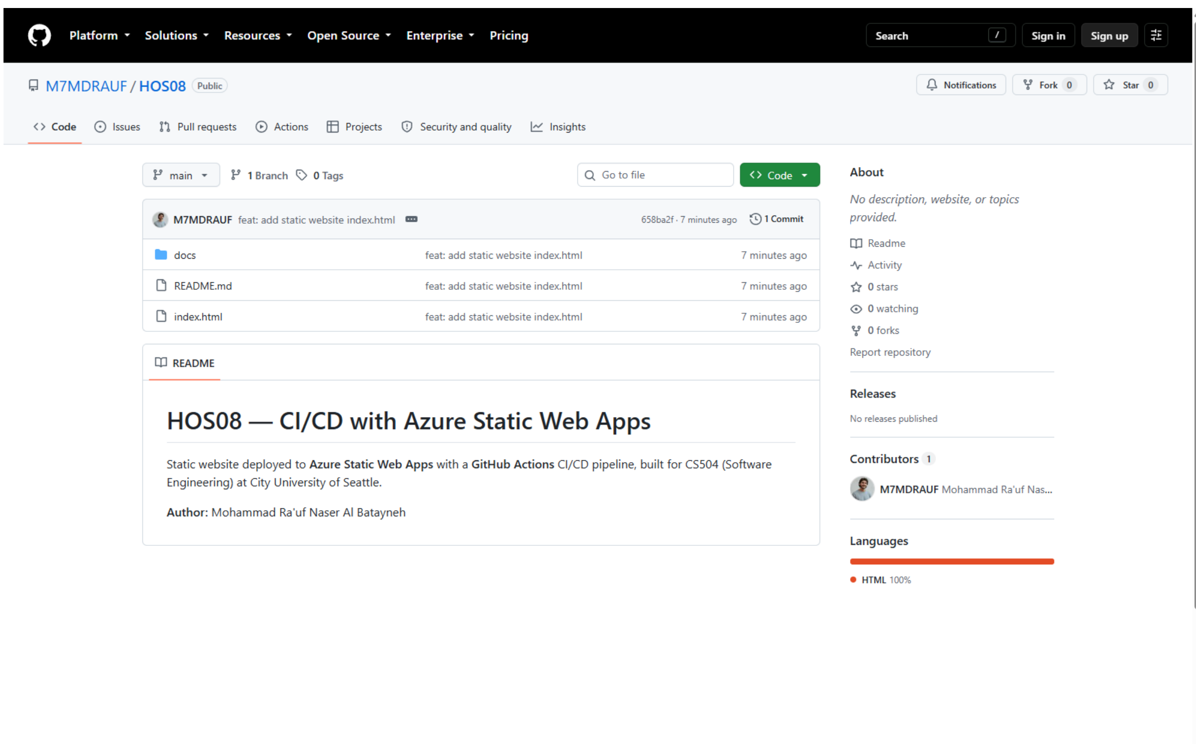

Figure 1 — The HOS08 repository on GitHub after pushing index.html to the main branch.

3. Part B — Provisioning the Azure Static Web App

3.1 Signing in to the Azure portal

The Azure portal was opened and signed in with the CityU account, under the City University of Seattle tenant. The account carries an Azure for Students Starter subscription, which is the no-cost student subscription used throughout this exercise.

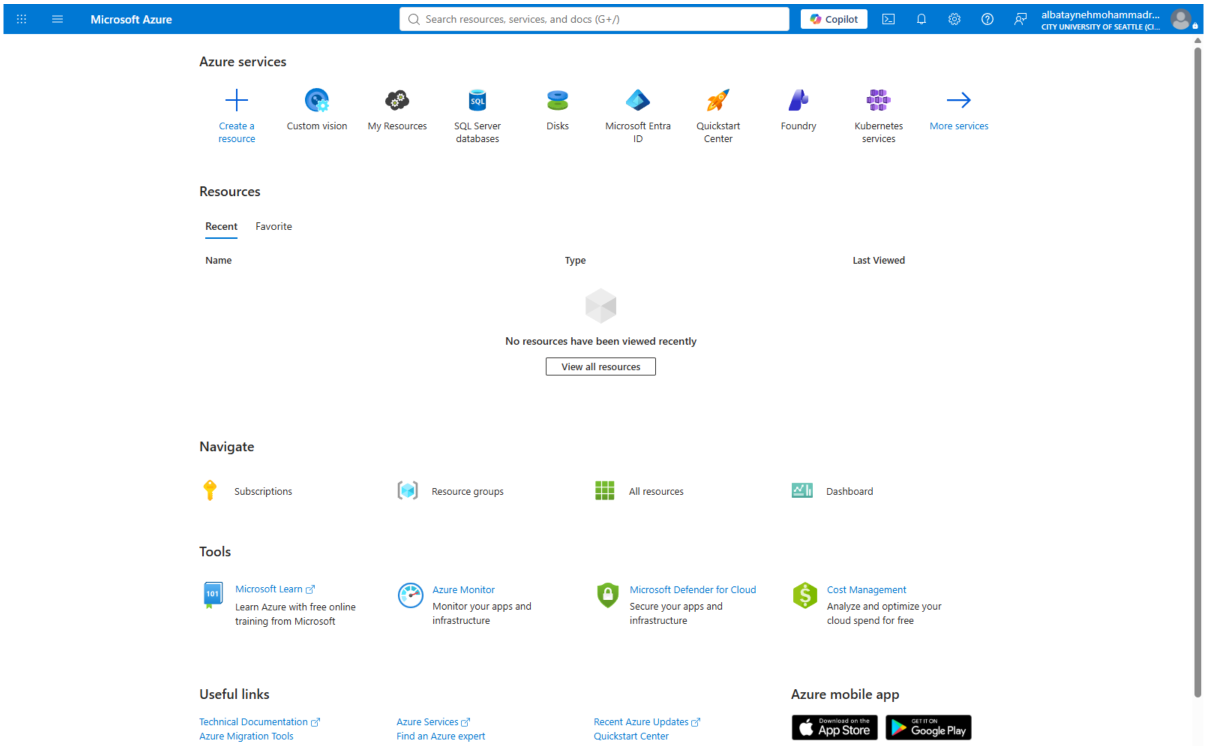

Figure 2 — The Azure portal home page, signed in under the City University of Seattle tenant.

3.2 Locating the Static Web Apps service

Typing "Static" into the portal search bar surfaces the Static Web Apps service.

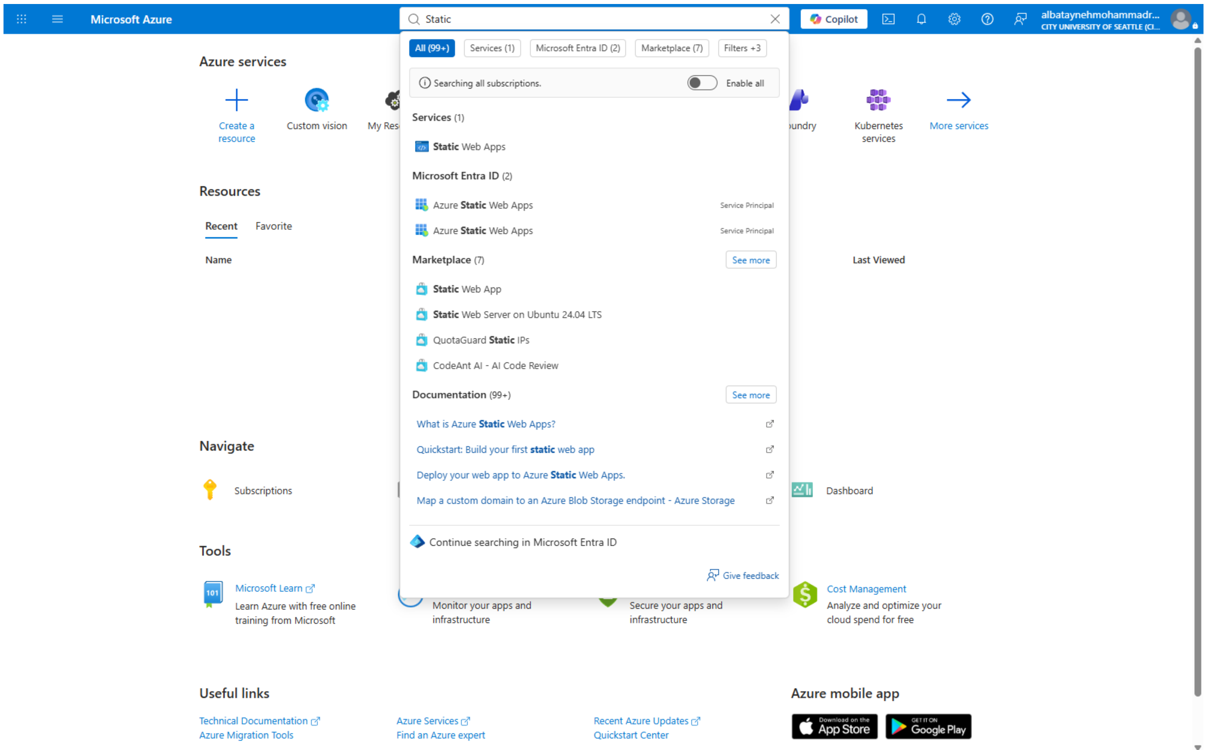

Figure 3 — Searching the portal for "Static" returns the Static Web Apps service.

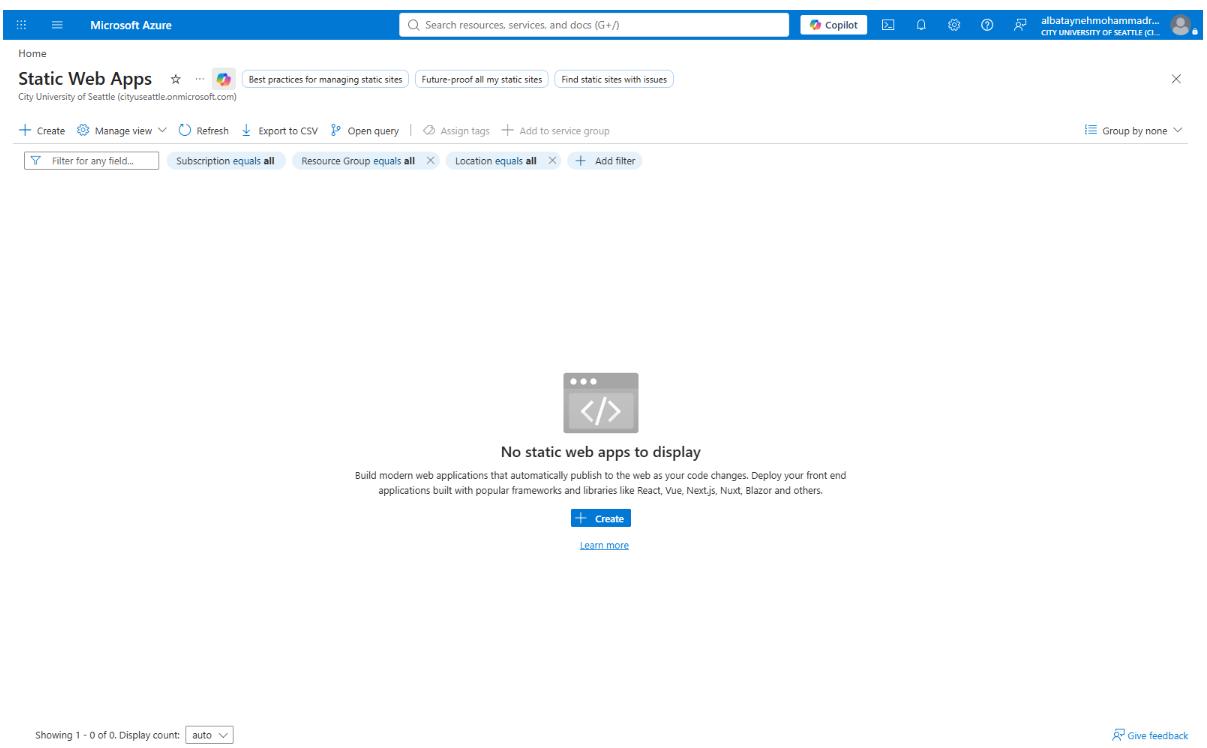

Figure 4 — The Static Web Apps blade before any resource exists.

3.3 Project details, name, and hosting plan

On the Basics tab, a new resource group named HOS08_group was created, the app was named HOS08, and the plan type was changed from the portal default of Standard to Free.

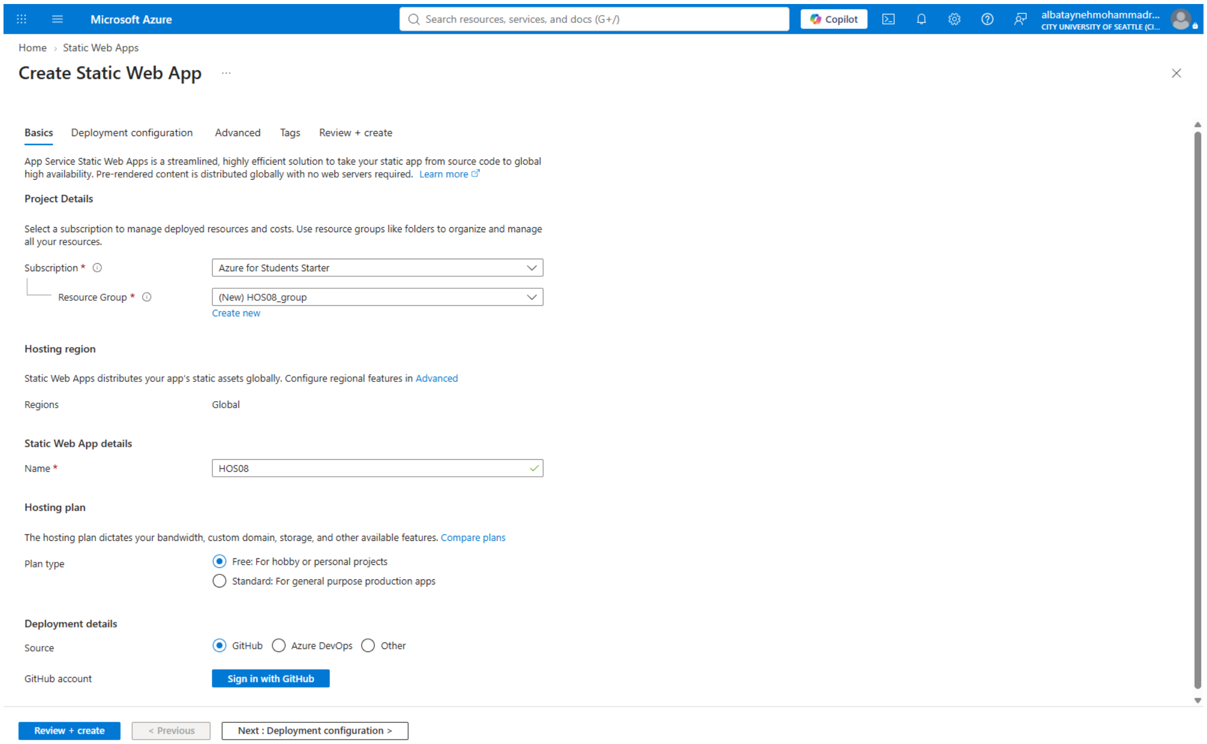

Figure 5 — The Basics tab with the resource group, name, and Free plan type set.

3.4 Connecting the GitHub account

Selecting GitHub as the deployment source and signing in authorises the Azure-App-Service-Static-Web-Apps OAuth application. Azure requests the "repo" and "workflow" scopes — it needs "repo" to read the repository and "workflow" to commit the pipeline definition into the .github/workflows directory on the student’s behalf.

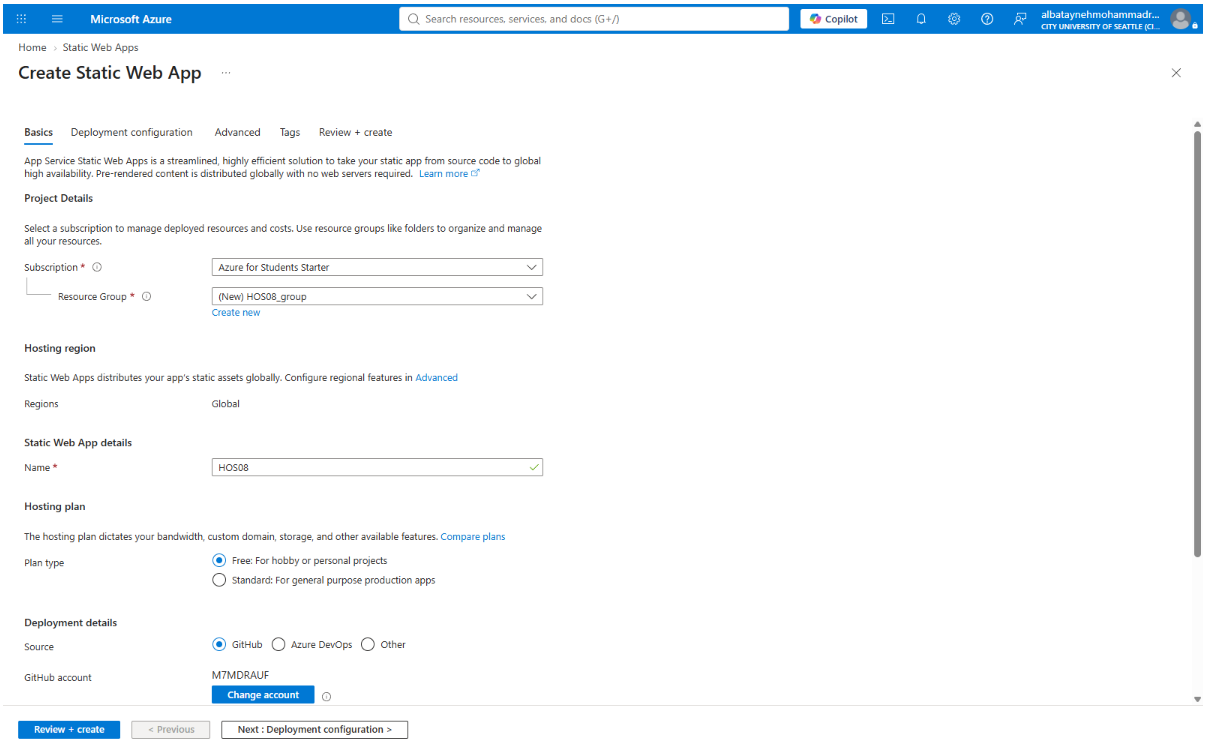

Figure 6 — The GitHub account M7MDRAUF connected to the deployment configuration.

3.5 Repository, branch, and build details

With the account connected, Azure populates the organisation, repository, and branch lists directly from GitHub. The HTML build preset was chosen because the site is plain HTML with no build step; app location and output location were both set to the repository root.

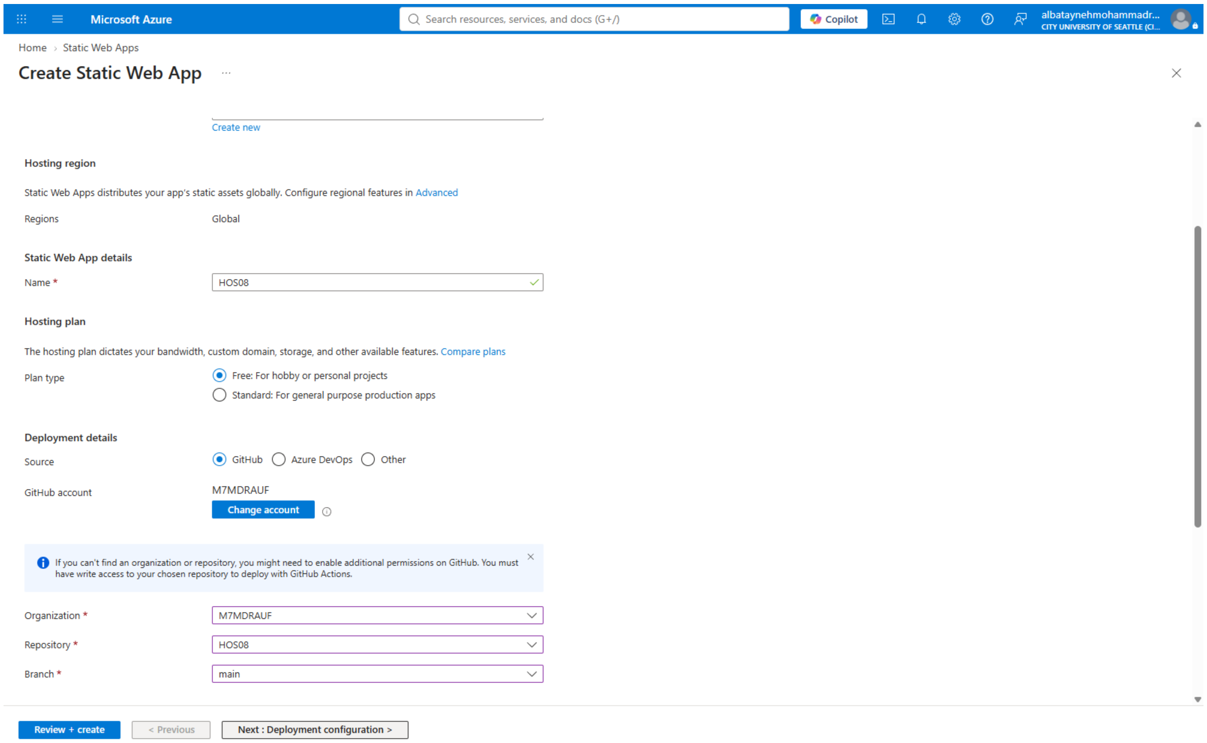

Figure 7 — Deployment details: organisation M7MDRAUF, repository HOS08, branch main.

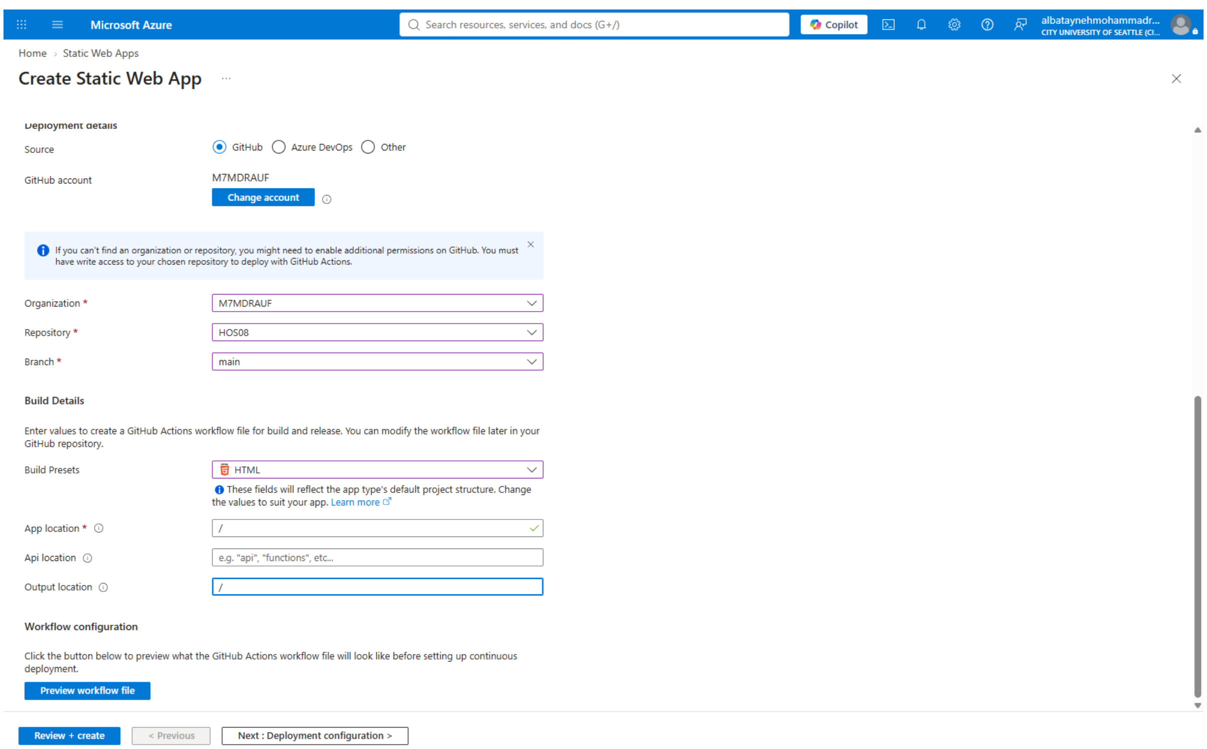

Figure 8 — Build details: HTML preset, app location "/", output location "/", API location left empty.

3.6 Previewing the generated workflow

The portal offers a preview of the GitHub Actions workflow file it is about to commit. Reading it before creating the resource is worthwhile — it makes explicit what will run, on which events, and which secret the deployment will authenticate with.

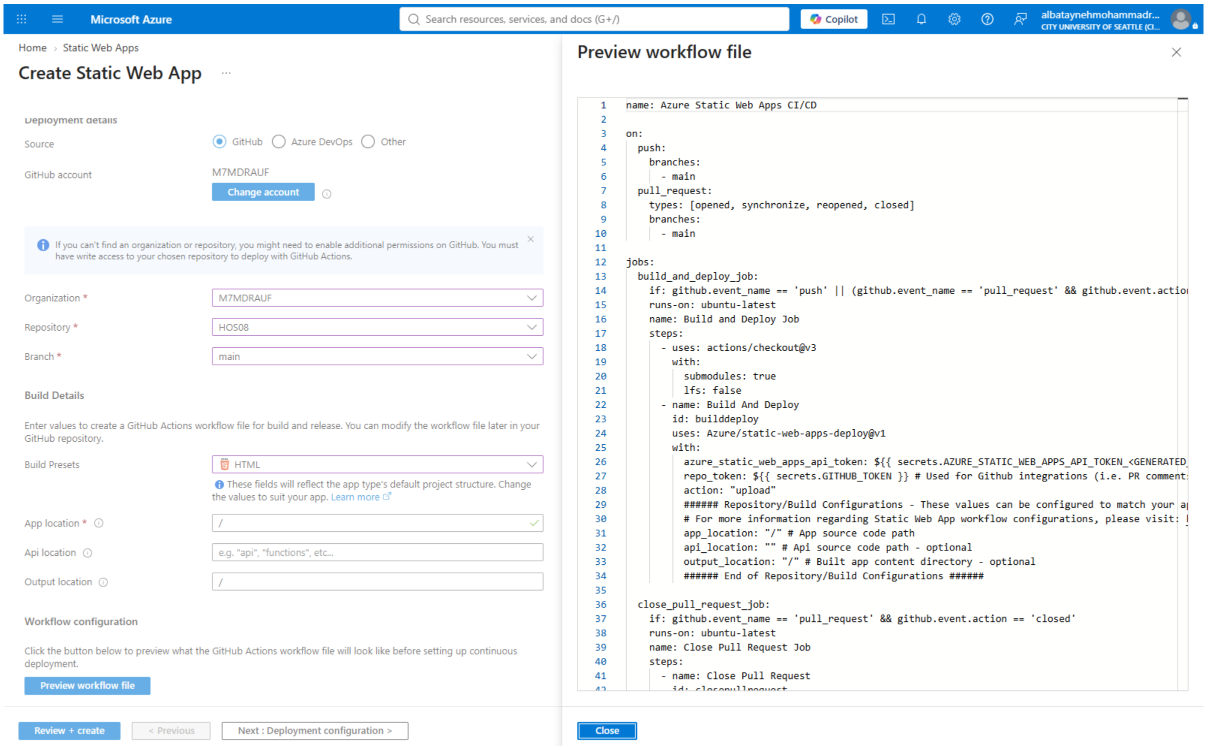

Figure 9 — Preview of the GitHub Actions workflow file Azure will commit to the repository.

4. A Validation Failure and How It Was Resolved

The first attempt at Review + create did not pass validation. The portal reported a policy violation rather than a configuration error, and the Create button stayed disabled.

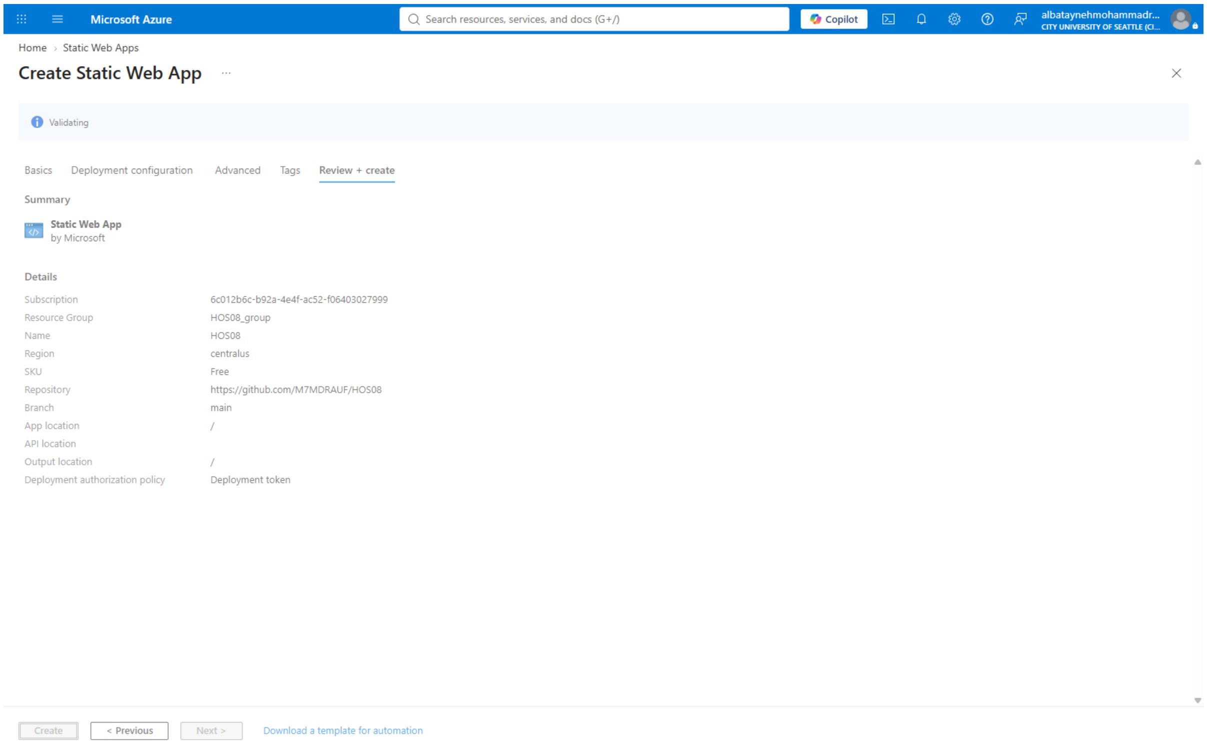

Figure 10 — The Review + create summary during validation.

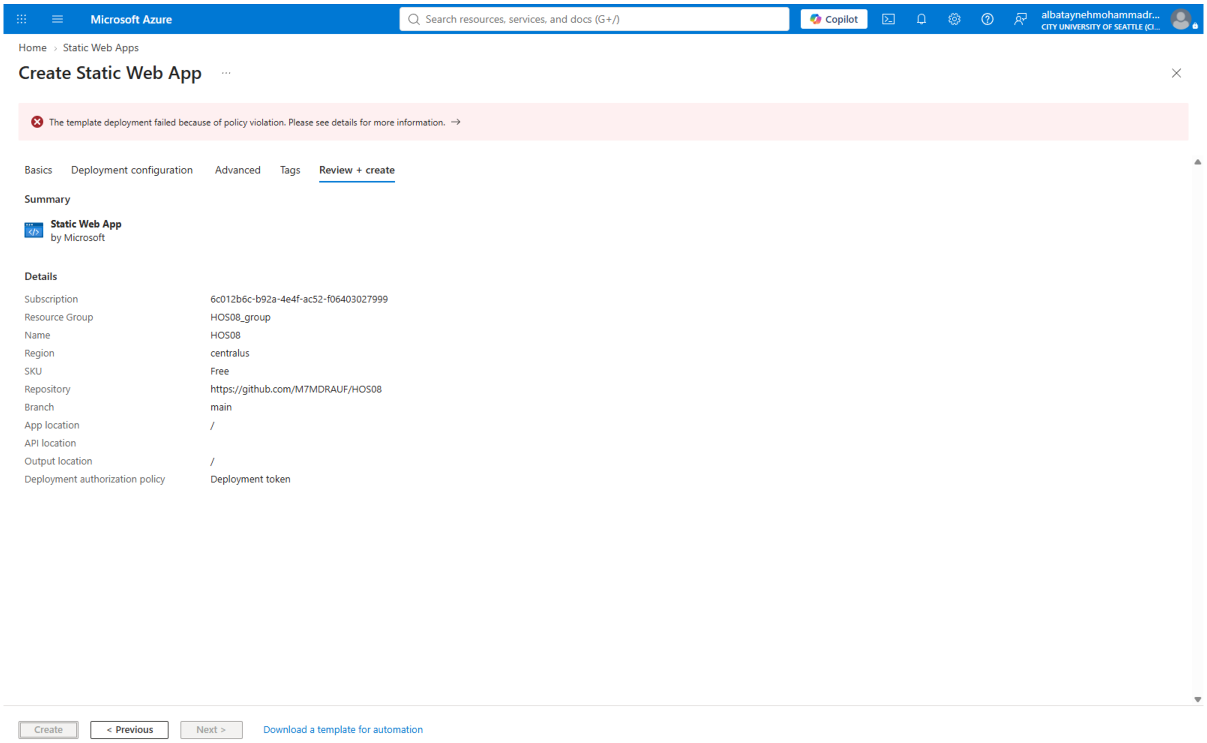

Figure 11 — Validation fails with a policy violation; the Create button remains disabled.

The full error text was:

```
Resource 'HOS08' was disallowed by Azure: This policy maintains a set of
best available regions where your subscription can deploy resources. The
objective of this policy is to ensure that your subscription has full
access to Azure services with optimal performance. Should you need
additional or different regions, contact support.
(Code: RequestDisallowedByAzure, Target: HOS08)
```

The obvious first guess — that the Free tier or the student subscription cannot host a Static Web App at all — is wrong. The message is about **regions**, not about tiers or quota. The Basics tab reports the hosting region only as "Global", which is easy to read as "there is no region to choose here". The region that actually matters is on the **Advanced** tab, under *Region for Azure Functions API and staging environments*, and the portal had defaulted it to Central US — a region this subscription's policy does not permit.

Setting that field to West US 2, the value the handout specifies, resolved the failure. This is a useful reminder that an Azure policy error names the constraint it enforced, and reading it literally is faster than guessing at plausible causes.

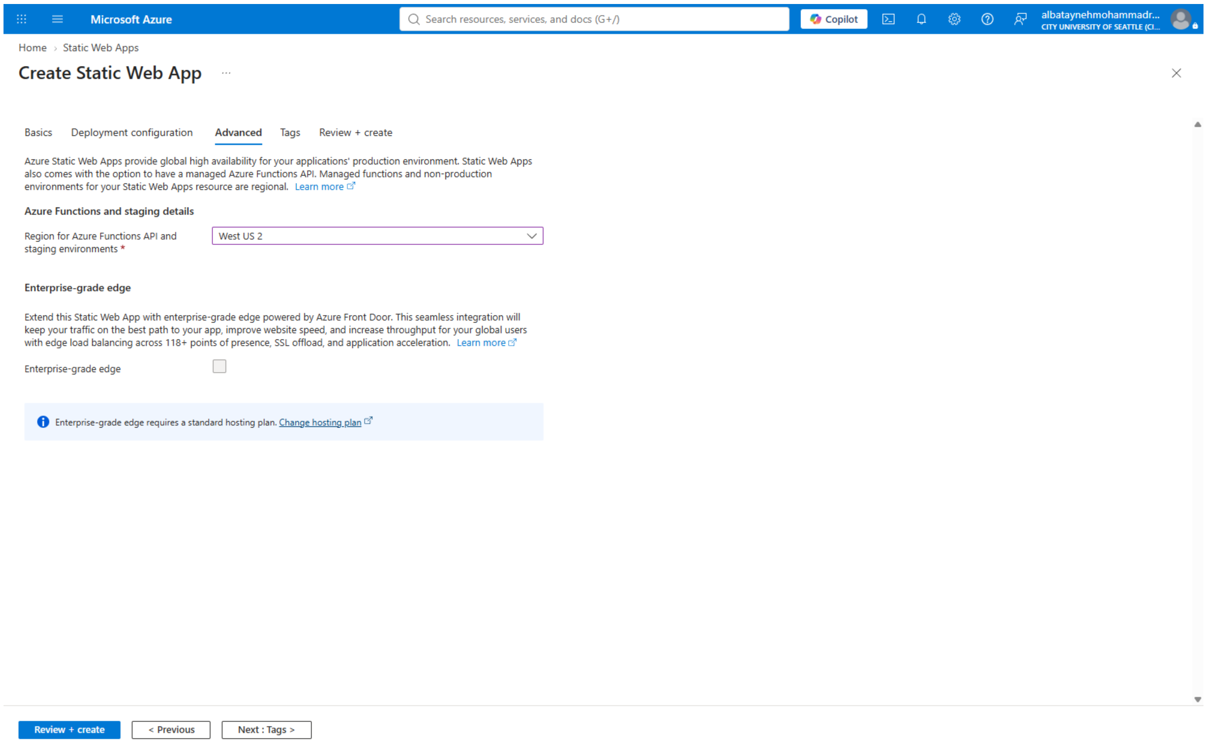

Figure 12 — The Advanced tab with the region changed from Central US to West US 2.

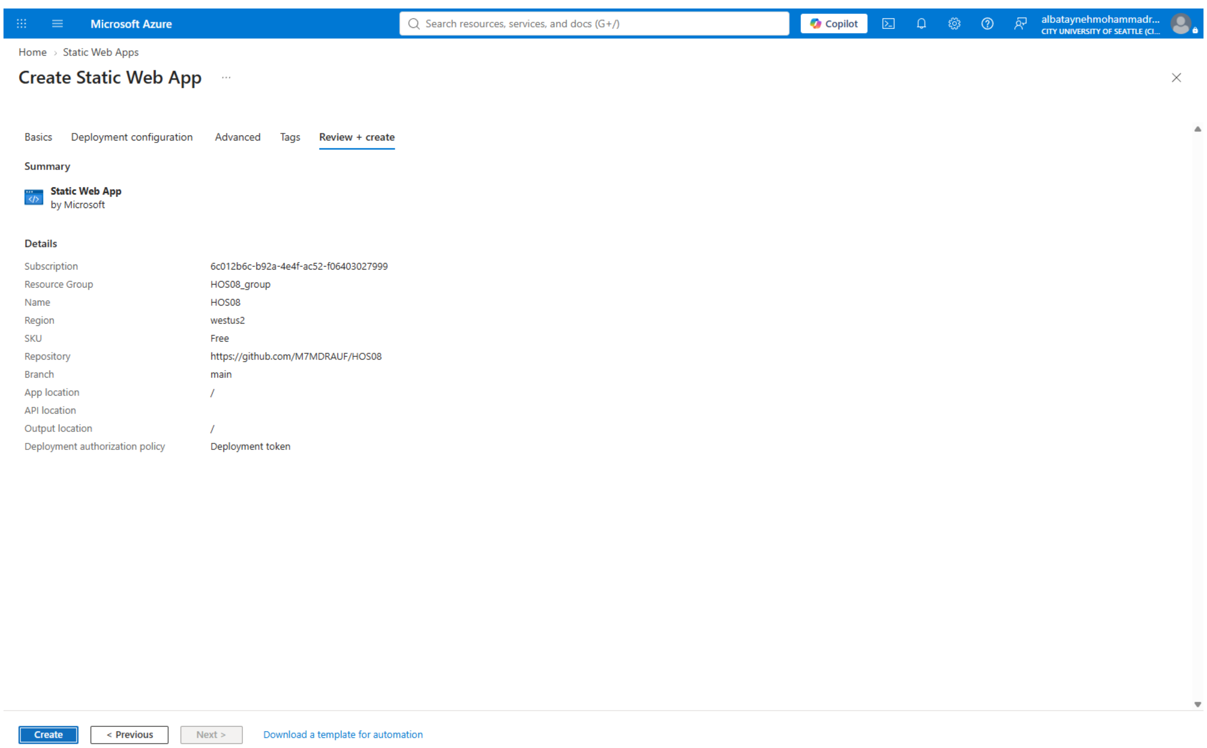

Figure 13 — Validation passes; SKU is Free, region is westus2, and Create is now enabled.

4.1 Resource created

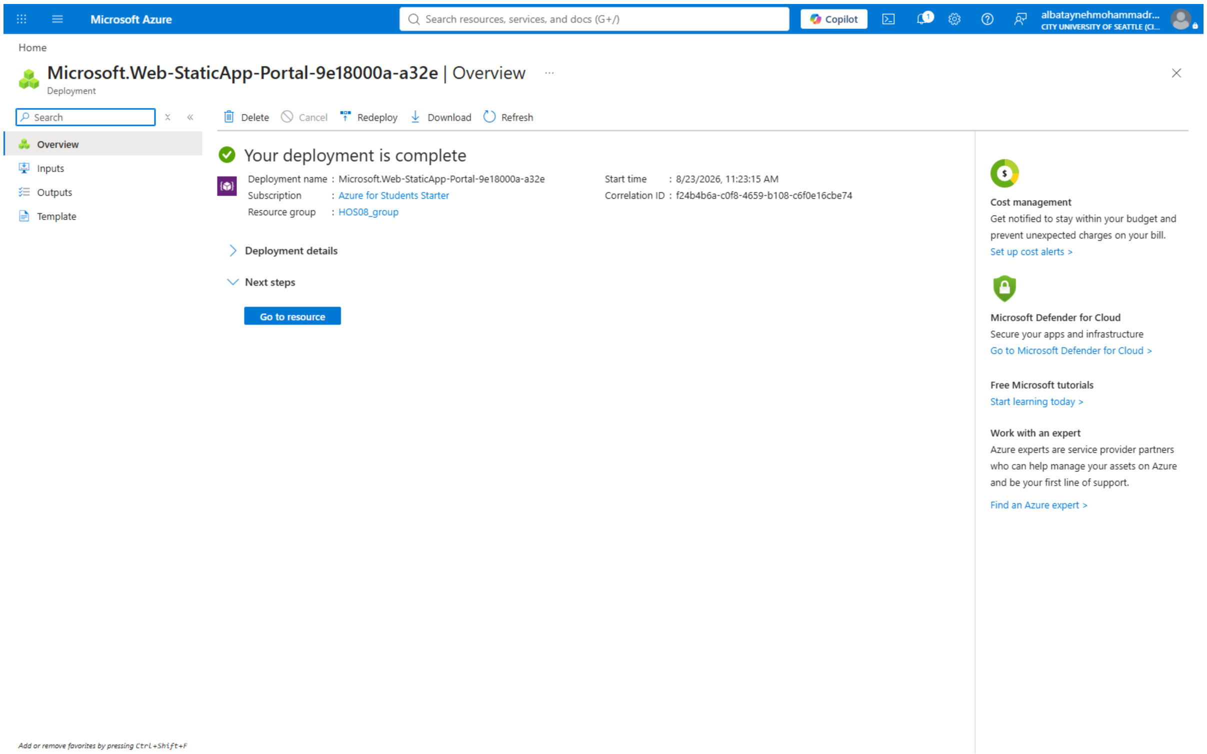

Figure 14 — Deployment succeeded — the HOS08 Static Web App is provisioned in HOS08_group.

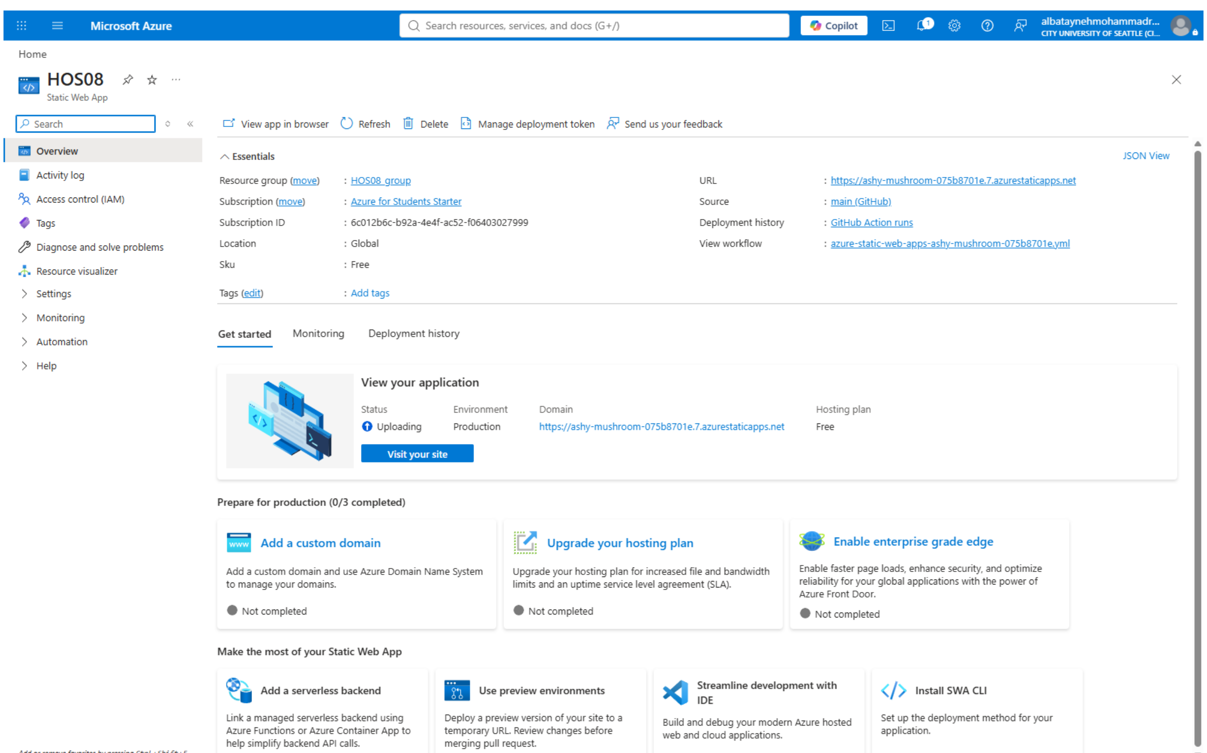

Figure 15 — The resource overview, showing the generated public URL and the GitHub source binding.

5. The Generated CI/CD Pipeline

On creation, Azure did two things to the repository. It committed a workflow file, .github/workflows/azure-static-web-apps-ashy-mushroom-075b8701e.yml, to the main branch, and it stored the deployment token as a repository secret. The pipeline therefore existed and ran before any further action was taken.

Job	Trigger	Purpose
build_and_deploy_job	Push to main; PR opened, synchronized, or reopened against main	Checks out the repository and runs Azure/static-web-apps-deploy@v1. A pull request deploys to a temporary staging environment; a push to main deploys to production.
close_pull_request_job	Pull request closed	Tears down the staging environment that was created for that pull request.

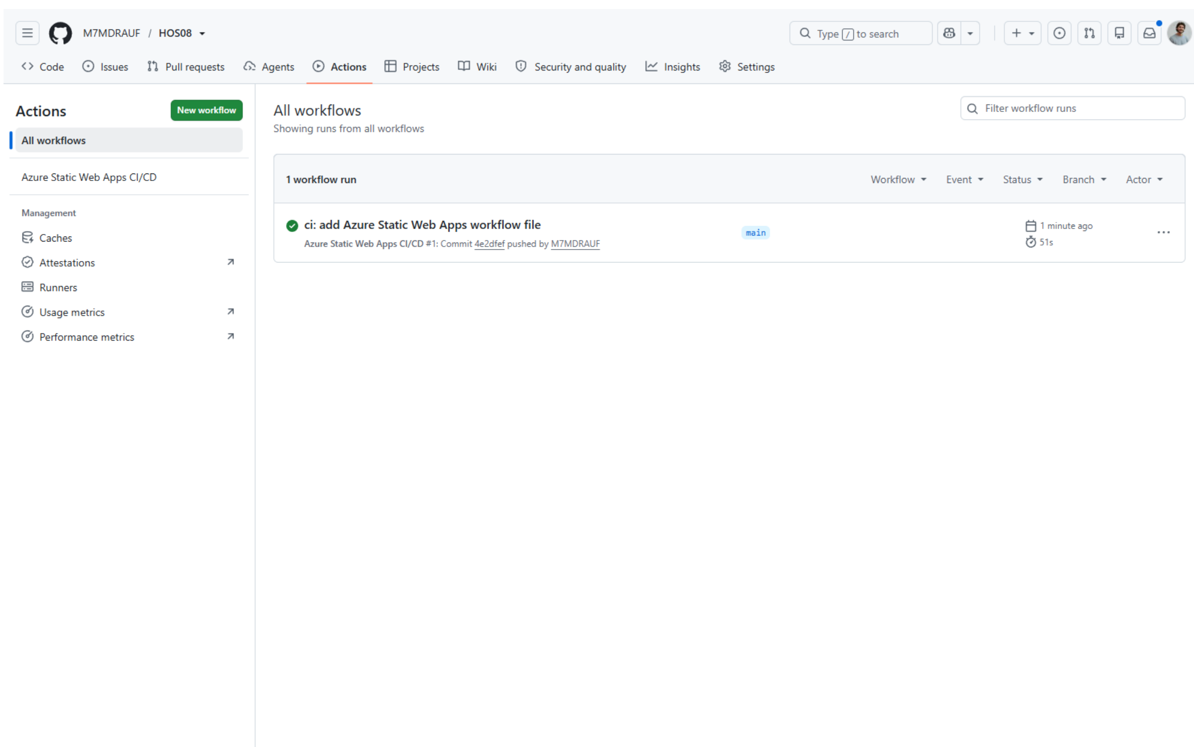

Figure 16 — The first workflow run, triggered by Azure committing the workflow file itself.

Opening the public URL at this point returns the original page, confirming that the pipeline deployed the site without any manual upload.

Figure 17 — The deployed site serving the original "Hello" page.

6. Part C — Exercising the Pipeline

A pipeline that has only ever deployed its own creation commit has not really been tested. The remaining steps push a genuine content change through the full branch-and-review workflow.

6.1 Creating the development branch

The workflow file Azure committed is pulled down first, then a development branch is created from main.

```
git pull
git checkout -b development
```

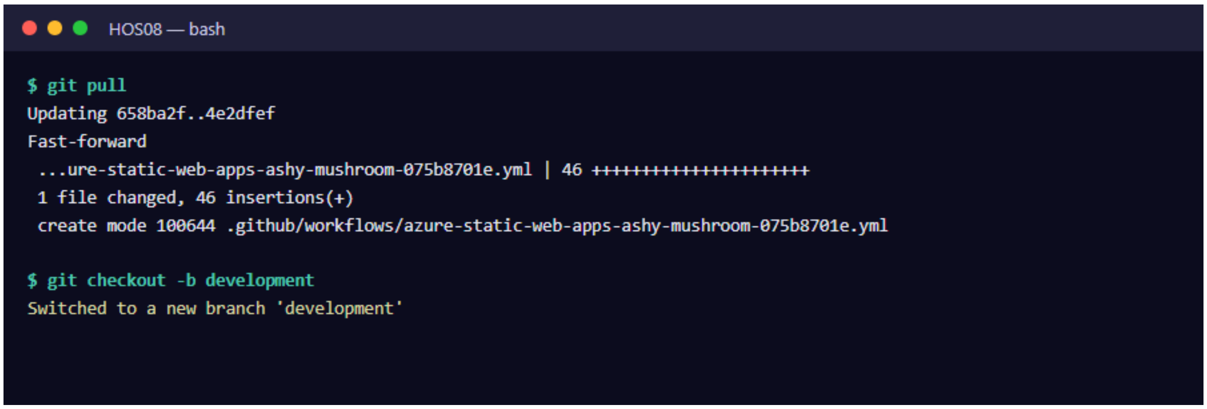

Figure 18 — Pulling the Azure-generated workflow file and branching to development.

6.2 Making and pushing the change

The heading in `index.html` was changed from `<h1>Hello</h1>` to `<h1>Hello World!</h1>`.

```
git add --all
git commit -m "development branch"
git push -f origin development
```

Figure 19 — Committing the change and pushing the development branch to GitHub.

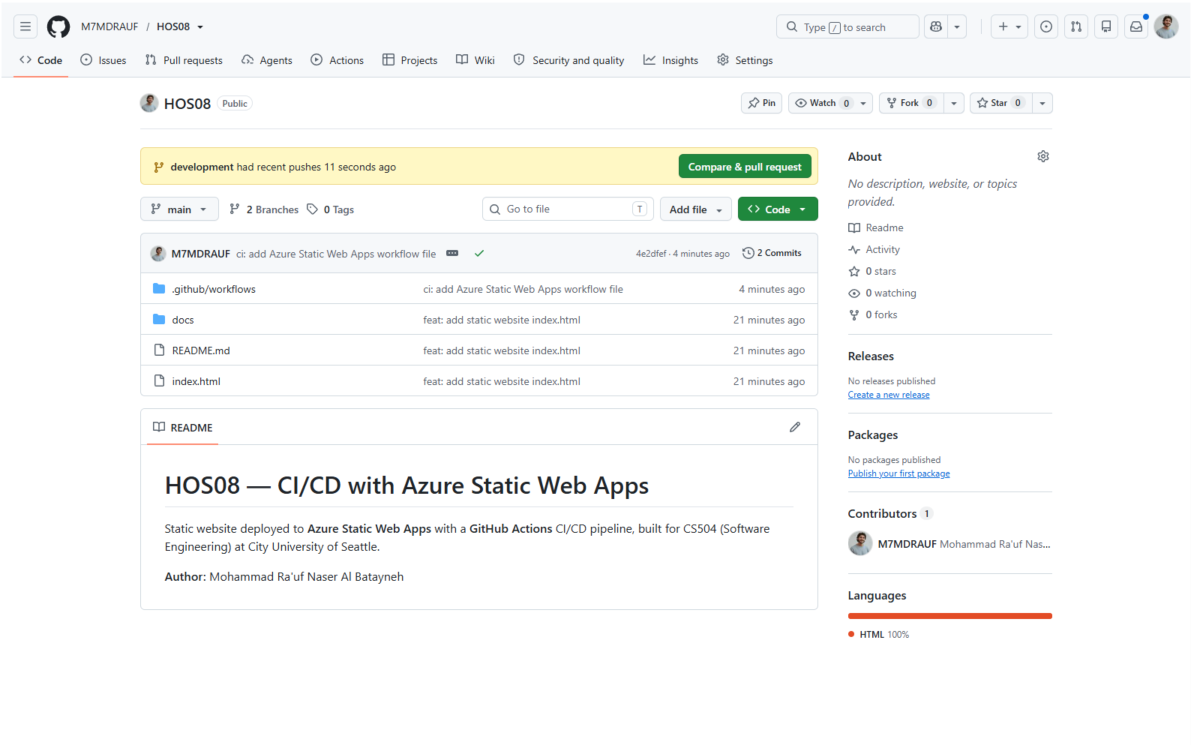

Figure 20 — GitHub detects the pushed branch and offers "Compare & pull request".

6.3 Pull request and preview environment

Opening the pull request triggers the pipeline a second time. Because the trigger is a pull request rather than a push to main, the workflow deploys to a temporary staging environment instead of production — the change can be reviewed on a real URL before it affects the live site. All checks passed.

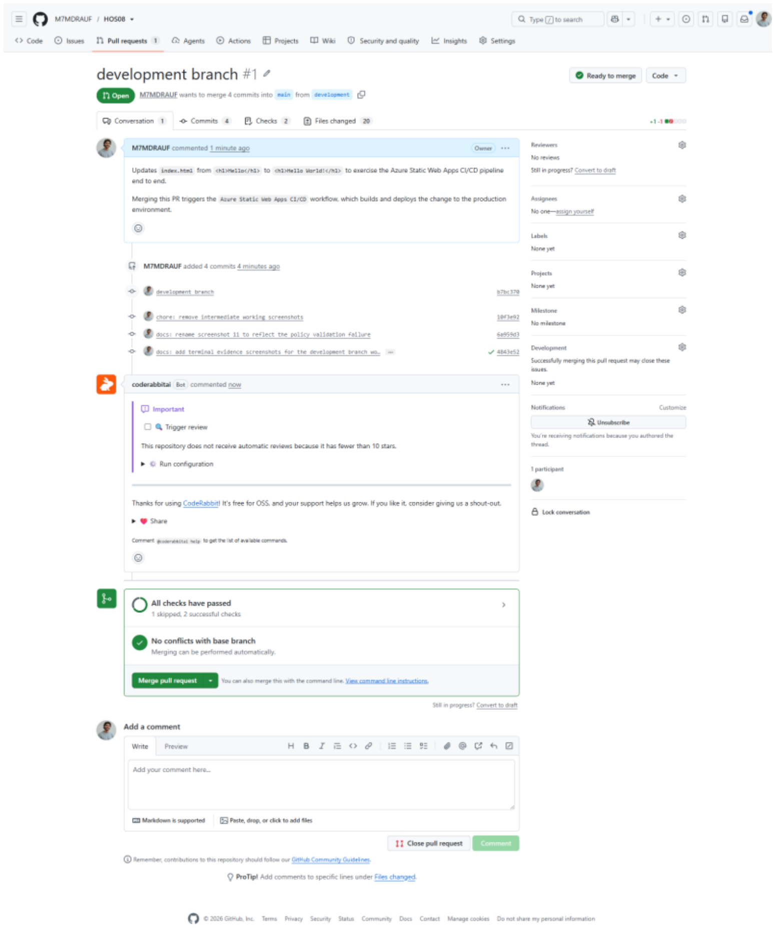

Figure 21 — Pull request #1 with all checks passed and no conflicts with the base branch.

6.4 Merging

Merging the pull request into main triggers the production deployment, and closing the pull request triggers the staging teardown job.

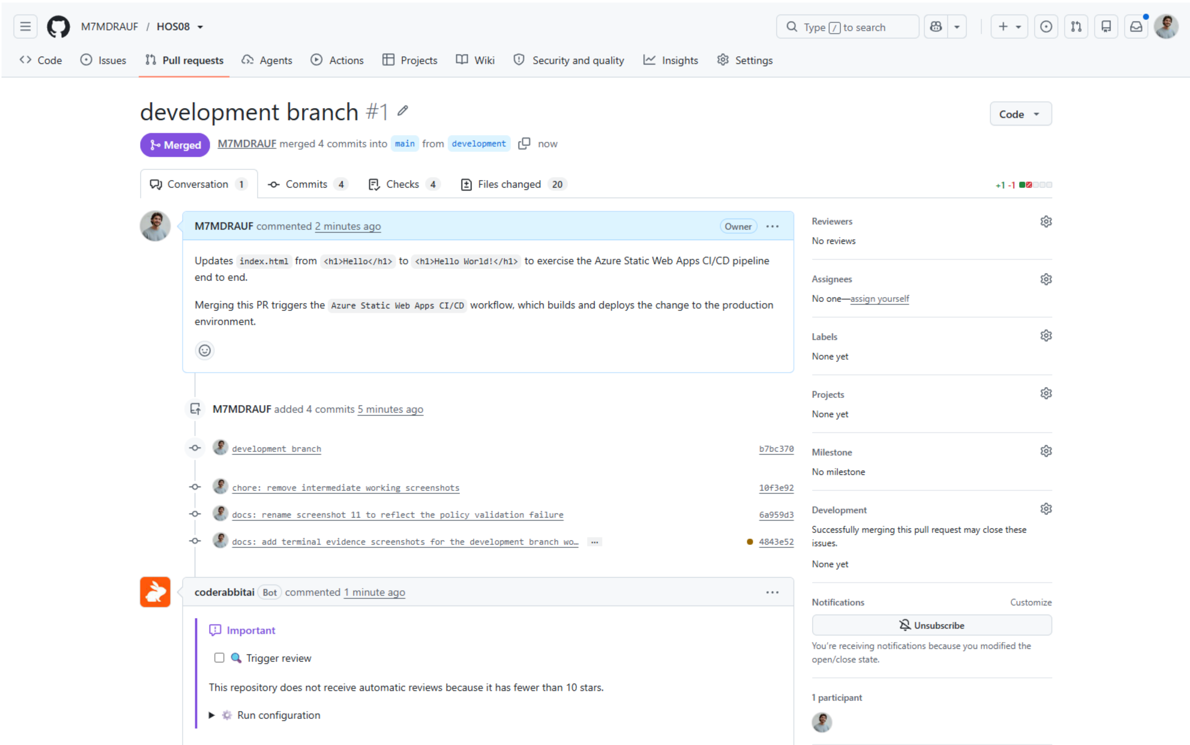

Figure 22 — Pull request #1 merged into main.

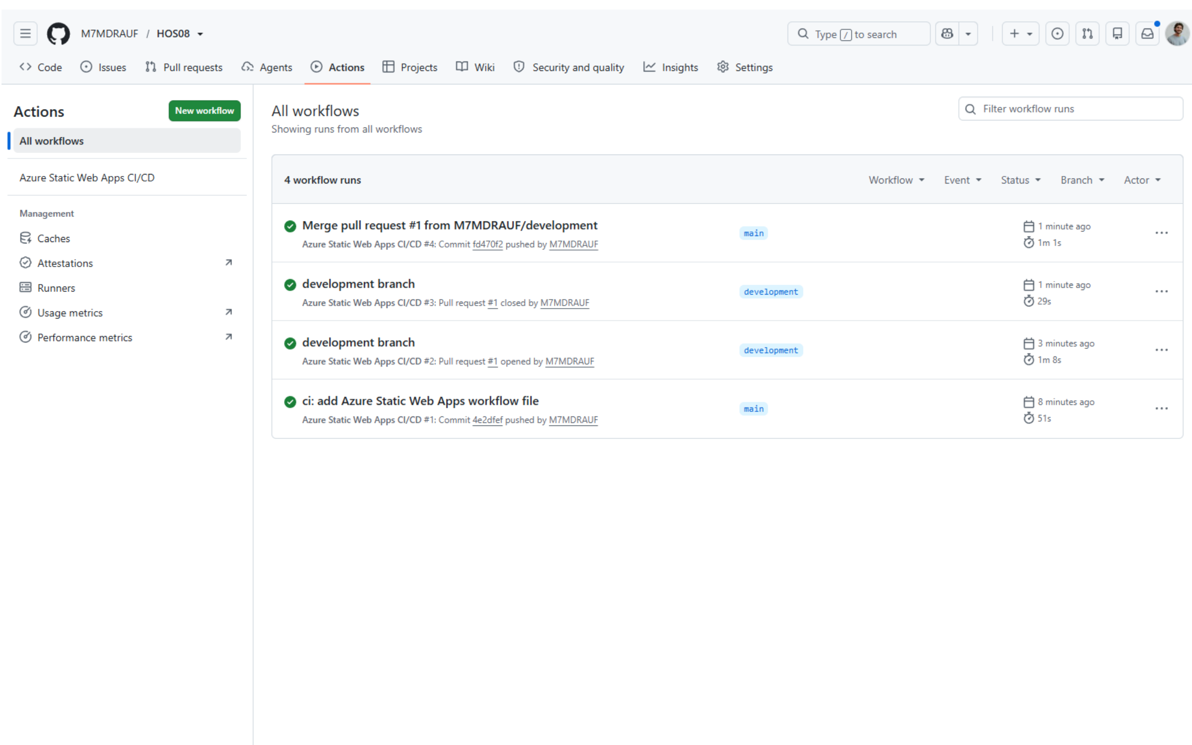

Figure 23 — All four workflow runs completed successfully.

#	Trigger	Branch	Duration	Result
1	Commit 4e2dfef — Azure adds the workflow file	main	51s	Success
2	Pull request #1 opened	development	1m 8s	Success
3	Pull request #1 closed	development	29s	Success
4	Merge commit fd470f2	main	1m 1s	Success

7. Verification

The Azure portal reports the production environment as Ready on the Free hosting plan, with main (GitHub) as its source.

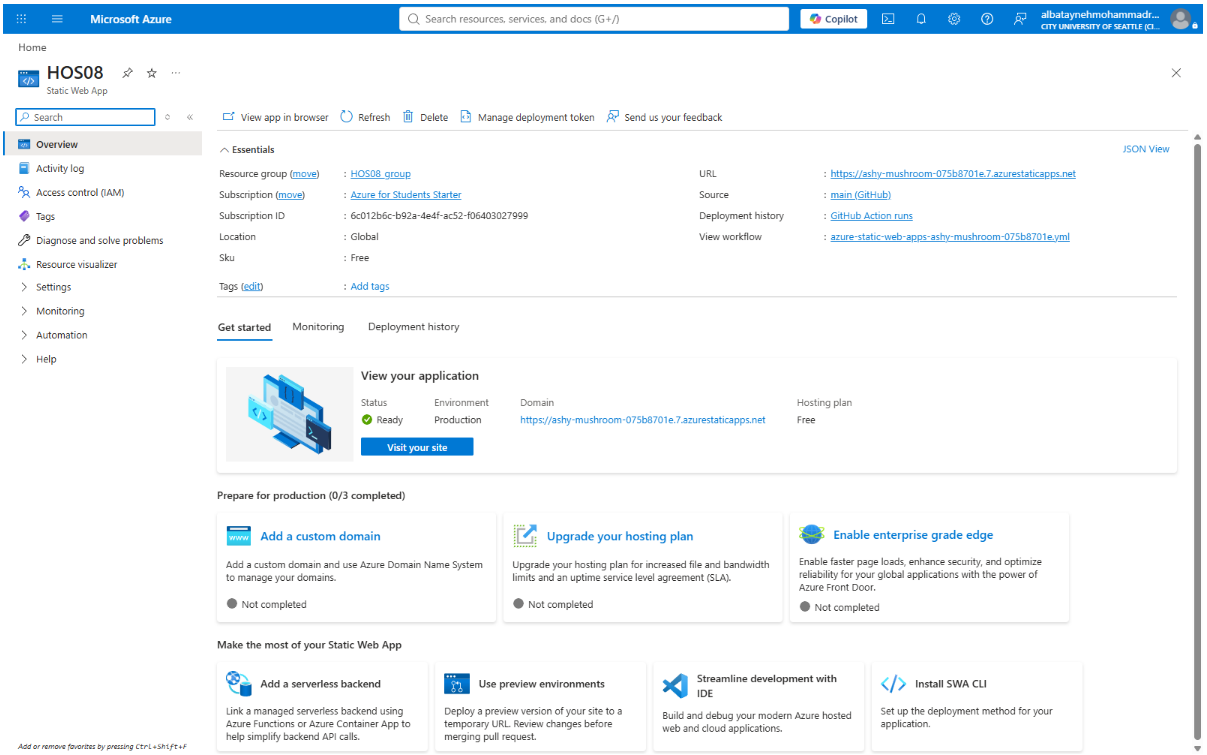

Figure 24 — The HOS08 resource: status Ready, environment Production, hosting plan Free.

Loading the public URL returns the updated heading. This is the result that matters: the text **Hello World!** was never uploaded to Azure by hand. It was committed to a branch, reviewed through a pull request, and merged — and the pipeline carried it to production on its own.

Figure 25 — The deployed site serving "Hello World!" after the merge — end-to-end proof of the pipeline.

8. Conclusion

The exercise produced a working deployment pipeline in which the repository, not a person, is the deployment mechanism. Four pipeline runs were observed, covering all three trigger types the workflow defines: a push to main, a pull request against main, and a pull request closing.

8.1 What this demonstrates about CI/CD

- The pipeline is defined in the repository. The workflow file is version-controlled alongside the code it deploys, so the deployment process is reviewable and reversible like any other change.
- Pull requests get their own environment. Reviewing a change on a real, isolated URL is far stronger evidence than reading a diff, and the staging environment is destroyed automatically when the pull request closes.
- Credentials never leave the platform. Azure stored the deployment token as a GitHub repository secret; the workflow references it by name and it is never present in the source.
- Deployment becomes uneventful. Once the pipeline exists, shipping is a merge — which is exactly the property that makes frequent, small releases practical.

8.2 What the failure taught

The policy violation was the most instructive part of the exercise. The error was specific and accurate, but it appeared on a tab that had not been visited, about a field whose relevance was not obvious from the Basics tab's "Global" region label. Reading the error text literally — it named regions, not tiers — pointed straight at the cause. Guessing at the more plausible-sounding explanation, that a student subscription cannot host the resource, would have led nowhere.

8.3 Cost management

The Free hosting plan was selected explicitly rather than accepting the portal's Standard default, and the summary on the Review + create tab was checked to confirm "SKU: Free" before provisioning. The handout also directs that the resource be deleted once the exercise is demonstrated, so that nothing continues to consume the subscription.

8.4 References

Microsoft. Azure Static Web Apps documentation. https://learn.microsoft.com/en-us/azure/static-web-apps/

Loubser, N. Software Engineering for Absolute Beginners: Your Guide to Creating Software Products. O'Reilly Online Learning.